A*(A-star) 알고리즘은 길찾기(경로 탐색) 문제에서 가장 널리 쓰이는 휴리스틱(Heuristic) 기반 탐색 알고리즘이며, 자율주행, 로봇 경로 계획, 게임 AI, 내비게이션 등에서 실제로 활용되고 있다.

참고 : 휴리스틱(Heuristic)이란 '경험적 추정' 또는 '직관적 판단 기준'을 의미다. 즉, 정확한 계산 대신 빠른 결정을 내리기 위해 사용하는 추정치나 규칙. A* 알고리즘에서 목표까지 남은 거리를 추정하여 탐색 효율을 높이는 경험적 판단 기준이다.

1. 정의

A* 알고리즘은 "현재까지 이동한 거리(G)"와 "목표까지의 예상 거리(H)"를 더한 총 비용(F)을 기준으로 가장 유망한 경로를 우선적으로 탐색하는 알고리즘이다.

$F(n)=G(n)+H(n)$

- G(n) 시작점에서 현재 노드까지 실제로 이동한 비용
- H(n) 현재 노드에서 목표까지의 추정(Heuristic) 비용
- F(n) 총 평가 함수 값 = G + H → 탐색 시 우선순위 큐 기준

2. 동작 원리

1) 시작 노드(start) 를 open list에 추가한다.
2) open list에서 F 값이 가장 작은 노드를 선택한다.
3) 그 노드의 인접 노드들을 탐색하면서 새로운 G, H, F 값을 계산하고 더 효율적인 경로라면 open list에 추가한다.
4) 선택한 노드가 목표(goal)에 도달하면 경로를 복원한다.
5) 그렇지 않으면 해당 노드를 closed list로 이동하고 2~4 반복.

3) 휴리스틱 함수 $H(n)$

- 목표까지의 거리를 "추정"하는 함수다.
- 완벽한 정답은 아니지만, 빠른 탐색을 유도한다.
- 대표적인 예:

  맨해튼 거리(Manhattan distance) : $|x_1-x_2| + |y_1-y_2|$

  유클리드 거리(Euclidean distance) : $\sqrt{(x_1-x_2)^2 + (y_1-y_2)^2}$

  대각선 거리(Diagonal distance) : $\max(|x_1-x_2|, |y_1-y_2|)$

* 휴리스틱이 낙관적(admissible) 이면 즉, 실제 비용보다 작거나 같다면, A*는 최단 경로(optimal path)를 보장한다.

4. 의사 코드 (Pseudo code)

```
open_set = PriorityQueue()
open_set.put((0, start))
came_from = {}
g = {start: 0}

while not open_set.empty():
    _, current = open_set.get()
```

```
    if current == goal:
        return reconstruct_path(came_from, current)

    for neighbor in neighbors(current):
        temp_g = g[current] + distance(current, neighbor)
        if neighbor not in g or temp_g < g[neighbor]:
            g[neighbor] = temp_g
            f = temp_g + heuristic(neighbor, goal)
            open_set.put((f, neighbor))
            came_from[neighbor] = current
```

5. 예시로 이해하기

예를 들어 격자형 지도에서 시작점(S)에서 목표(G)까지 이동한다고 하면, A*는 다음과 같은 전략을 쓴다.

"지금까지 실제로 온 거리(G)가 짧고, 앞으로 남은 거리(H)도 적을 것 같은 방향을 우선 탐색하자."

→ 즉, 단순 최단거리 탐색(Dijkstra) 의 현실적 효율성과 탐색 방향 유도(Heuristic) 를 결합한 것이 A*이다.

6. Dijkstra vs A*

구분	**Dijkstra**	**A***
탐색 기준	G(n) (실제 이동 거리)	G(n) + H(n) (예상 거리 포함)
휴리스틱 사용	X 없음	O 있음
탐색 효율	느림 (모든 방향 탐색)	빠름 (목표 방향으로 유도)
최적 경로 보장	O	O (단, H가 낙관적일 때만)

7. 자율주행에서의 A*

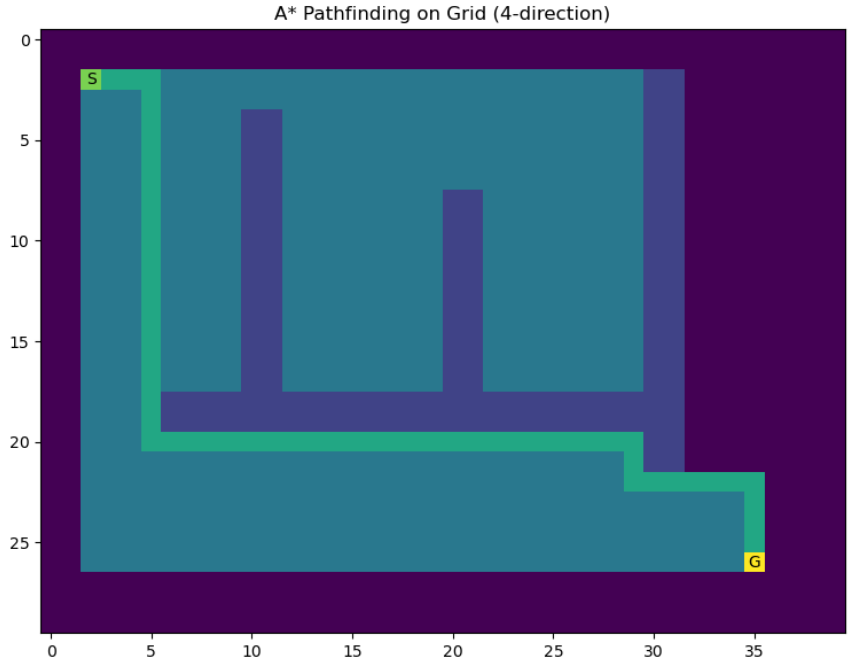
자율주행 시뮬레이터에서는 도로의 격자화된 지도(grid map)나 노드 그래프 상에서 장애물(Obstacle)을 피하며, A* 로 경로 계획(path planning) 을 수행한다. 그 후, A*가 찾은 경로(waypoints)를 따라가는 제어는 PID 제어기, Pure Pursuit, 또는 강화학습 정책(DQN 등)이 담당한다. 즉, A* 는 "어디로 갈지"를 DQN/제어기 는 "어떻게 움직일지"를 결정한다.

* 요약표

항목	**내용**
알고리즘 이름	A*
핵심 개념	G(실제 비용) + H(추정 비용) = F
장점	빠른 탐색, 최적 경로 보장 (H가 낙관적일 때)
단점	메모리 사용 많음, 큰 맵에서 느릴 수 있음
주요 활용	자율주행, 로봇 내비게이션, 게임 AI, 경로 탐색

~~~ A* 알고리즘을 이용한 간단한 2D 경로 시각화 파이썬 예제 (matplotlib 출력) : 별첨 ~~~
~~~

예제 실행 결과 ----------



이 그림은 A* 알고리즘(A-star) 으로 2차원 격자 지도 위에서 시작점(S) 에서 목표점(G) 까지 최단 경로를 탐색한 결과를 시각화한 것이다. A* 알고리즘이 장애물을 피해, 가장 짧은 경로로 S에서 G까지 이동하는 과정을 성공적으로 수행한 결과를 시각화 함.

~ 그림 설명 ~

- 보라색(가장 어두운 부분) → 이동할 수 없는 장애물(Obstacle)
- 파란색 배경 → 탐색 가능한 빈 공간(Free cell)
- 초록색 경로 → A* 알고리즘이 계산해 낸 최단 경로(Path)
- S → 시작(Start) 지점
- G → 목표(Goal) 지점

~ 알고리즘 동작 요약 ~

1) A*는 각 셀을 노드로 보고, 상하좌우(4방향)로 이동할 수 있는 격자에서 탐색을 수행한다.
2) 이동할 때마다 F(n) = G(n) + H(n) 값을 계산.
    - G(n): 시작점에서 현재 위치까지 실제 이동 거리
    - H(n): 현재 위치에서 목표까지의 추정 거리(여기서는 맨해튼 거리)
3) F값이 가장 작은 노드를 우선적으로 확장하면서, 장애물을 피해 최적의 경로를 찾아간다.
4) 그림에서 보이는 초록색 선이 바로 A*가 탐색해 낸 최단 경로이다.